

1. Forward elimination algorithm

```
~/Documents/lab07
> ./elimination
Original System [A | b]:
      2      -1      1 |      8
      4       1     -1 |     13
      2      -1      3 |     16

Upper Triangular Form:
      2      -1      1 |      8
      0       3     -3 |     -3
      0       0      2 |      8
```

2. Backward substitution

```
~/Documents/lab07
> g++ -o sub substitution.cpp

~/Documents/lab07
> ./sub
Solving the Upper Triangular System...

Solution:
Variable x_0 = 3.5
Variable x_1 = 3
Variable x_2 = 4
```

2. Big-Theta (n^2)

- 1 The outer loop iterates from $i = n - 1$ down to 0.
- 2 For each iteration i , the inner loop iterates from $j = i + 1$ to $n - 1$. The number of operations in the inner loop is exactly $n - 1 - i$.
- 3 To find the total number of operations $T(n)$, we sum the inner loop executions over all outer loop iterations:

$$T(n) = \sum_{i=0}^{n-1} (n - 1 - i)$$

Let $k = n - 1 - i$. As i goes from 0 to $n - 1$, k goes from $n - 1$ down to 0. This allows us to rewrite the sum as a standard arithmetic progression:

$$T(n) = \sum_{k=0}^{n-1} k = \frac{(n-1)n}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

Because the highest-order term is quadratic and there are no conditional early exits, the algorithm is bounded both above and below by a quadratic function. Therefore, the running time is strictly $\Theta(n^2)$.

3. Random array into heap

```
~/Documents/lab07  
> g++ -o maxheap maxheap.cpp  
  
~/Documents/lab07  
> ./maxheap  
Original array:  
4 10 3 5 1 9 2  
  
Array transformed into a Max-Heap:  
10 5 9 4 1 3 2
```

4. LU decomposition based on Gaussian elimination

```
Original Matrix A:  
    2    -1    1  
    4     1   -1  
    2    -1    3  
  
Lower Triangular Matrix L:  
    1     0     0  
    2     1     0  
    1     0     1  
  
Upper Triangular Matrix U:  
    2    -1    1  
    0     3   -3  
    0     0    2
```